

Simulation Environment Versioning Module (SEVM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: SIMULOS® — Simulation Governance Architecture

Parent Standard: Simulation Model & Environment Standard (SMES)

Operational Layer: Simulation Model & Environment Governance Layer

Category: Governance & Enforcement

Subcategory: Simulation Governance

Type: Parent Standard Module

Governed Space: Simulation Environment Versioning

Version: 1.0

Status: Canonical · Open Module

Effective Date: 17 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · CLIA® · MGIA™ · ASGA™ · RIS™

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Governed Space

Simulation Environment Versioning

Canonical Definition

Simulation Environment Versioning Module (SEVM) defines the governance framework for identifying, establishing, distinguishing, freezing, changing, comparing, preserving, restoring, and reconstructing the versions and materially relevant configuration states of a governed Simulation Environment.

SEVM establishes the version-control layer through which simulation executions remain bound to the exact environmental configuration under which they occurred, preventing materially different Simulation Environments from being treated as if they were the same simulation context.

Operational Role

SEVM is the fifth and final internal governance space of the Simulation Model & Environment Standard (SMES).

The preceding modules establish:

SMMM — What is the Simulation Model structurally composed of?

SECM — Within what configured Simulation Environment does that model operate?

SBIM — How do the model, environment, and connected domains interact across their boundaries?

SPHM — Which controlled parameters configure the model and environment for simulation activity?

SEVM asks:

Which exact version of the Simulation Environment existed for a particular simulation execution, what changed between environment versions, and can the environment state relevant to that execution later be reconstructed?

The governed progression is:

Model Structure → Environment Configuration → Boundary Interaction →
Parameterization → Environment Version Identification → Version Baseline →
Change Control → Execution Binding → Version Reconstruction → Governed
Simulation Environment Version

Module Operational Space

SEVM governs: Simulation Environment version identity, environment version identifiers, environment configuration states, environment baselines, environment snapshots, version composition, version lineage, version ancestry, version succession, version differentiation, version equivalence, version compatibility, version activation, version deactivation, version supersession, version freezing, version restoration, version rollback, environment forks, environment branches, environment merges, environment change, material change classification, environment configuration drift, dependency version change, boundary configuration change, parameter configuration change references, environment release states, execution-to-version binding, output-to-version traceability, version comparability, historical environment preservation, point-in-time environment reconstruction, and environment-version lifecycle continuity.

Module Operational Scope

SEVM applies wherever a change in the Simulation Environment may materially alter simulation behavior, execution conditions, comparability, repeatability, interpretation, or resulting Simulation Outputs.

It may apply across: engineering simulations, robotics simulations, autonomous-system simulations, AI simulations, digital twins, scientific simulations, industrial simulations, infrastructure simulations, transportation simulations, aerospace simulations, mission simulations, financial simulations, environmental simulations, cyber-physical simulations, multi-agent simulations, distributed simulations, federated simulations, software-in-the-loop environments, hardware-in-the-loop environments, cloud-based simulation platforms, live-system-connected simulations, and future simulation environments.

Governance depth should remain proportional to the material effect environment change may have upon the simulation inquiry.

Module Function

A simulation environment is rarely static.

Software changes.

Libraries change.

Dependencies change.

External services change.

Hardware changes.

Boundary configurations change.

Environmental assets change.

Parameter baselines change.

Models connected to the environment change.

If these changes are not version-governed, two executions may appear to use the same simulation while actually operating under materially different conditions.

SEVM exists to prevent: unidentified environment versions, silent environment updates, loss of historical configurations, execution against unknown environment states, uncontrolled dependency changes, environment drift, inappropriate comparison across materially different versions, loss of reproducibility, ambiguous rollback, and inability to reconstruct the environment that produced a simulation result.

Simulation Environment Version

A Simulation Environment Version is an identifiable governed state of the Simulation Environment representing a materially distinguishable configuration applicable during a defined simulation context.

Conceptually:

Simulation Environment + Governed Configuration State + Version Identity =
Simulation Environment Version Version Identity

Each materially relevant Simulation Environment Version should be distinguishable.

A Version Identity may include: Environment Identifier, Version Identifier, configuration reference, release identifier, activation date, dependency state, parameter baseline reference, boundary configuration reference, and applicable model compatibility.

Version Identifier

A Version Identifier provides a persistent reference to a specific governed environment state.

For example:

SEV-1.0

SEV-1.1

SEV-2.0

The numbering mechanism may differ by implementation.

The governance requirement is not a particular numbering convention.

The requirement is:

materially distinguishable environment states must remain distinguishable.

Environment Version Composition

A Simulation Environment Version may be composed of multiple governed components.

Conceptually:

Environment Version = Environment Configuration + Dependencies + Boundary Configuration + Parameter Baseline + Relevant Runtime State

Where these components have independent versions, their relationships should remain traceable.

Composite Version Identity

Complex Simulation Environments may require a Composite Version Identity.

For example:

Environment Version 4.2

may consist of:

Simulation Engine 7.1

Physics Library 3.4

Sensor Package 2.8

Weather Model 5.0

Boundary Configuration B12

Parameter Baseline P17

The composite relationship should remain reconstructable where material.

Environment Baseline

An Environment Baseline establishes an approved reference state of the Simulation Environment.

The baseline provides a controlled point against which subsequent change may be evaluated.

Conceptually:

Baseline Environment → Change → Successor Environment Version Baseline Content

A material Environment Baseline may identify: environment structure, software components, dependency versions, relevant hardware, environmental assets, interfaces, boundary configurations, parameter references, runtime configuration, external service dependencies, and other materially relevant environment properties.

Baseline ≠ Permanent Environment

A baseline is a governed reference state.

It does not prohibit change.

It makes change visible.

Environment Snapshot

An Environment Snapshot captures the materially relevant environment state at a defined point in time or execution context.

Snapshots may support: forensic reconstruction, repeatability, rollback, comparison, incident analysis, and execution traceability.

Snapshot ≠ Version

A snapshot records a state.

A version represents a governed environment identity.

Multiple snapshots may exist within one version where transient runtime state differs without constituting a new governed version.

Version Lineage

SEVM should preserve the lineage between environment versions.

Conceptually:

**Version 1.0 → Version 1.1 → Version 1.2 →
Version 2.0**

Lineage allows later determination of: predecessor, successor, change path, branching, rollback, and historical dependency.

Version Ancestry

Where environments are derived from earlier configurations, ancestry should remain visible.

For example:

Environment B v1.0 derived from Environment A v3.2

This may be important where assumptions, dependencies, or configuration defects propagate between versions.

Environment Version Change

An Environment Version Change occurs when a materially relevant property of the governed Simulation Environment changes.

Possible changes include: simulation engine update, dependency update, library update, hardware replacement, environment asset change, boundary configuration change, external service change, parameter baseline change, timing-system change, synchronization change, network configuration change, or runtime architecture change.

Material Version Change

Not every technical modification requires a new governed environment version.

The governing question is:

Could the change materially affect simulation behavior, execution conditions, repeatability, comparability, interpretation, or reconstruction?

If yes, the change should be visible to environment-version governance.

Version Change Classification

Environment changes may be classified as:

Non-Material

Minor

Material

Major

Critical

or another implementation-specific classification.

Classification should reflect simulation consequence rather than software-release terminology alone.

No Silent Version Change Principle

A materially changed Simulation Environment must not continue to be represented as the same governed environment version merely because the technical system retained the same name or deployment location.

Version Differentiation

Two environment states should be considered materially different where their differences can materially alter simulation behavior or interpretation.

Version differentiation may depend upon: changed dependencies, changed interaction behavior, changed parameterization, changed timing, changed physical components, changed external services, changed environmental assets, or changed runtime conditions.

Version Equivalence

Two technical environment instances may be treated as version-equivalent only where their materially relevant configuration states are sufficiently equivalent for the simulation inquiry.

Technical duplication does not automatically establish governance equivalence.

Environment Compatibility

A Simulation Environment Version may be compatible only with certain: Simulation Model versions, parameter configurations, boundary configurations, scenario structures, execution engines, hardware, or dependencies.

Compatibility should remain identifiable where incompatibility could materially alter execution. Model–Environment Version Compatibility

A Simulation Model Version should not be assumed compatible with every Simulation Environment Version.

Conceptually:

Model Version + Environment Version → Compatibility Status

Possible statuses may include:

Compatible

Conditionally Compatible

Incompatible

Compatibility Undetermined Dependency Versioning

A Simulation Environment may rely upon external or internal dependencies.

Examples include: software libraries, physics engines, databases, AI models, APIs, external services, hardware firmware, operating systems, middleware, or

communication infrastructure.

Material dependency versions should remain identifiable.

Dependency Change

A dependency change may create a new environment state even where the primary Simulation Environment software remains unchanged.

For example:

Same Simulation Platform + New Physics Engine = Potentially Different
Simulation Environment External Service Version Change

External services may change outside direct simulation control.

Where such change materially affects simulation behavior, the affected environment state should not remain invisible.

SEVM may preserve:

External Service Identity → Previous State → Changed State → Affected
Environment Version Boundary Configuration Version Relationship

SBIM governs Simulation Boundary Interaction.

SEVM preserves which governed boundary configuration applied to a particular Simulation Environment Version.

Conceptually:

Environment Version → Boundary Configuration Reference

A material boundary change may therefore require environment-version reassessment.

Parameter Configuration Version Relationship

SPHM governs Simulation Parameterization.

SEVM preserves the applicable Parameter Baseline or configuration relationship where it forms part of the environment version.

Conceptually:

Environment Version → Parameter Baseline Reference

Not every parameter change requires a new Environment Version.

But a material baseline-level configuration change may.

Version Activation

A Simulation Environment Version should have an identifiable activation point where necessary.

This allows determination of which version was operational for a specific simulation execution.

Version Deactivation

When an environment version is no longer approved for new execution, its deactivation should remain distinguishable from deletion.

Historical versions may still be required for reconstruction.

Version Supersession

A new environment version may supersede a prior version.

Conceptually:

SEV-1.0 → Superseded by → SEV-1.1

Supersession does not erase the historical environment state.

Version Freeze

A Version Freeze prevents material changes to an environment configuration during a defined simulation campaign, controlled comparison, benchmark, investigation, or validation activity.

The purpose is to maintain stable execution conditions.

Frozen Environment Principle

A frozen Simulation Environment Version must not materially change during the governed freeze period without creating an explicit exception or successor version.

Environment Release State

An environment version may hold a governed release state such as:

Draft

Test

Approved

Active

Frozen

Deprecated

Superseded

Archived

or another implementation-specific state.

Release state should not be confused with validation status. Environment Version $\neq$ Validation State

SEVM identifies and governs Simulation Environment Versions.

VALIDOS® governs validation conclusions and Validation States.

Therefore:

Environment Version 3.0

does not mean:

Validated Environment Version 3.0

unless a separate validation process establishes that state.

Environment Version Fork

A Version Fork occurs when one governed environment state develops into multiple successor configurations.

For example:

Environment 2.0

↓

Environment 2.1-A

and

Environment 2.1-B

Forks should preserve common ancestry.

Environment Branch

Branches may support: experimental environments, alternative configurations, hardware variants, regional configurations, different scenario families, or research variants.

A branch should not silently replace the primary environment baseline.

Environment Merge

Where branches are recombined, the resulting environment should identify the material configuration inherited from each branch.

Merge complexity should not destroy version traceability.

Environment Rollback

A Rollback restores a previous governed environment configuration.

Rollback should identify: current version, target version, reason, affected executions, activation point, and any retained changes.

Rollback ≠ Erasure

A rollback does not erase the intervening version history.

Conceptually:

V1 → V2 → Rollback to V1-derived state

should preserve the existence and use of V2.

Environment Restoration

Restoration may reconstruct a historical environment for: repeat execution, forensic analysis, validation, comparison, or incident investigation.

The restored environment should remain distinguishable from the historical original where differences remain.

Restoration Fidelity

A restored environment may be:

Exact

Functionally Equivalent

Partially Reconstructed

or:

Reconstruction Uncertain

depending upon what can actually be restored.

This distinction should remain explicit.

Environment Configuration Drift

Environment Configuration Drift occurs where the actual Simulation Environment progressively departs from its governed version or baseline without corresponding version governance.

Examples include: silent dependency updates, changed runtime settings,

modified hardware, external service changes, unrecorded configuration changes, parameter baseline changes, or altered boundary behavior.

Drift Detection

SEVM should support detection of material divergence between:

Declared Environment Version

and:

Actual Environment State Declared–Actual Version Consistency

A simulation execution should not claim:

Environment Version X

when the actual materially relevant environment state no longer corresponds to Version X.

Version Integrity Interface

SEVM does not own general integrity governance.

Where preservation, tamper resistance, authenticity, or record integrity requires dedicated controls, INTEGROS® may apply.

SEVM governs:

which version existed and what it contained.

INTEGROS® may govern:

whether the relevant version records and artifacts preserved their required integrity.

Execution-to-Version Binding

Every material Simulation Execution should be bindable to the Simulation Environment Version under which it occurred.

Conceptually:

Simulation Execution ID → Simulation Environment Version ID

This relationship is fundamental to repeatability and interpretation.

Execution Environment Record

An execution record may preserve: Execution Identifier, Environment Version Identifier, Model Version reference, Parameter Configuration reference, Boundary Configuration reference, dependency state, execution time, and relevant environment snapshot.

No Unbound Execution Principle

A material Simulation Execution should not produce governable Simulation Output without an identifiable relationship to the Simulation Environment Version under which that execution occurred.

Output-to-Version Traceability

Downstream Simulation Output should remain traceable to the environment version that contributed to its production.

Conceptually:

Environment Version → Simulation Execution → Simulation Output

SEVM establishes the environment-version side of this chain.

Version Comparability

Two simulation executions should not be assumed comparable merely because they used the same Simulation Model.

Their environment versions may differ.

SEVM therefore asks:

Are the materially relevant environment conditions sufficiently equivalent for the intended comparison?

Version Comparison

A Version Comparison may identify: common components, changed components, changed dependencies, changed boundaries, changed parameter baselines, changed runtime conditions, and expected behavioral consequences.

Comparability Status

Possible statuses may include:

Environment-Comparable

Conditionally Environment-Comparable

Partially Environment-Comparable

Not Environment-Comparable

or:

Environment Comparability Undetermined Version Delta

A Version Delta represents the material differences between two Simulation Environment Versions.

Conceptually:

Version B – Version A = Environment Version Delta

The delta may identify: added components, removed components, updated components, dependency changes, boundary changes, parameter baseline changes, runtime changes, and configuration changes.

Version Delta Propagation

A material Environment Version Delta may affect later SIMULOS® governance spaces.

Conceptually:

Environment Version Change

↓

Input / Scenario Conditions

↓

Simulation Execution

↓

Simulation Output

↓

Reality Correlation

↓

Simulation Evidence

SEVM preserves the version change.

Downstream Parent Standards govern their respective consequences.

Environment Version Continuity

SEVM should preserve the lifecycle:

Environment Creation → Baseline → Active Version → Change → Successor Version
→ Supersession / Branch / Rollback → Archive

This allows the environment to remain governable across repeated simulation activity.

Historical Environment Preservation

Historical environment versions may be necessary to answer:

Which environment produced this result?

What changed afterward?

Could the execution be repeated?

Did a later environment update alter behavior?

Was the relevant defect present at the time of execution?

Point-in-Time Environment State

SEVM should support determination of the environment state applicable at a specific point in time.

This is particularly important for: incident reconstruction, scientific reproducibility, regulatory review, certification support, validation, dispute resolution, and audit.

Point-in-Time Reconstruction

A Point-in-Time Reconstruction should establish, where possible:

Environment Identity

Environment Version

Configuration State

Dependencies

Boundary Configuration

Parameter Baseline

Runtime State

and:

Execution Relationship

for the relevant time.

Environment Version Reconstruction

SEVM should enable reconstruction sufficient to answer:

Which environment version was active?

What did that version contain?

Which dependencies were present?

Which boundary configuration applied?

Which parameter baseline applied?

What changed from the previous version?

Was the version frozen, active, deprecated, or superseded?

Which executions used it?

Can the relevant state be restored?

Version Traceability

A complete version traceability path may be represented as:

Simulation Environment

↓

Environment Baseline

↓

Environment Version

↓

Environment Configuration

↓

Dependency State

↓

Boundary Configuration

↓

Parameter Baseline



Simulation Execution



Simulation Output

Minimum Implementation Framework

1. Identify the Simulation Environment

Establish the governed environment whose versions require control. 2.

Establish Environment Version Identity

Assign distinguishable version references to materially different environment states. 3. Establish the Environment Baseline

Define the controlled reference configuration. 4. Capture Material Version Composition

Preserve materially relevant components, dependencies, boundaries, and parameter references. 5. Govern Environment Change

Determine whether changes require a successor version or other governed treatment. 6. Preserve Version Lineage

Maintain predecessor, successor, branch, merge, rollback, and ancestry relationships. 7. Control Activation, Freeze, Supersession, and Restoration

Ensure environment lifecycle states remain explicit. 8. Detect Material Configuration Drift

Identify divergence between the declared version and actual environment state.

9. Bind Simulation Execution to Environment Version

Preserve the exact environment-version relationship for material executions.

10. Preserve Historical Reconstruction

Maintain sufficient information to compare, restore, and reconstruct materially relevant environment states.

Governance Outputs

SEVM may produce: Simulation Environment Version Identifier, Environment Version Record, Environment Baseline, Environment Snapshot, Environment Version Composition Record, Composite Version Identity, Version Lineage Record, Version Ancestry Record, Environment Version Change Record, Material Change Classification, Environment Version Delta, Environment Version Comparison, Environment Comparability Status, Model–Environment Compatibility Record, Dependency Version Record, External Service Version Record, Boundary Configuration Version Reference, Parameter Baseline Version Reference, Environment Activation Record, Environment Deactivation Record, Version

Supersession Record, Version Freeze Record, Environment Release State, Environment Fork Record, Environment Branch Record, Environment Merge Record, Environment Rollback Record, Environment Restoration Record, Restoration Fidelity Record, Environment Configuration Drift Record, Declared–Actual Version Consistency Record, Execution Environment Record, Execution-to-Version Binding, Output-to-Version Traceability Reference, Point-in-Time Environment State, Environment Version Reconstruction Record, Historical Environment Record, and Simulation Environment Version Traceability Map.

Use Case 1 — Autonomous Vehicle Simulation Platform

Scenario

An autonomous vehicle simulation is executed using:

Environment Version 4.2

containing a defined physics engine, road environment, sensor emulation system, weather model, dependency set, boundary configuration, and parameter baseline.

The physics engine is later updated.

Application

SEVM evaluates the change as material because vehicle behavior may change.

A successor environment becomes:

Environment Version 4.3

rather than silently retaining Version 4.2. Result

Results generated under 4.2 and 4.3 remain distinguishable, and any comparison can account for the environment change. Use Case 2 — Space Mission Simulation

Scenario

A mission simulation produces a critical navigation result.

Months later, a defect is discovered in a communication-model dependency.

Application

SEVM traces the affected dependency across environment-version lineage and identifies which executions used the defective version.

The historical environment is reconstructed. Result

Investigators can determine whether the critical simulation result was produced under the affected environment rather than rerunning the mission unknowingly under a corrected configuration. Use Case 3 — Scientific Climate Simulation

Scenario

A long-running scientific simulation program produces results across several years.

During that period: libraries change, computational infrastructure changes, parameter baselines evolve, and external datasets are updated.

Application

SEVM preserves distinct governed Simulation Environment Versions and binds each execution to its corresponding environment state. Result

Researchers can distinguish scientific differences caused by changed simulation assumptions or scenarios from differences potentially introduced by changes in the Simulation Environment itself.

Architectural Position

Simulation Environment Versioning is the fifth and final internal governance space of the Simulation Model & Environment Standard (SMES).

The complete SMES progression is:

Simulation Model Structure → Simulation Environment Configuration → Simulation Boundary Interaction → Simulation Parameterization → Simulation Environment Versioning

Together, the five modules establish:

what the Simulation Model contains,

where it operates,

how it interacts across its boundaries,

how its configurable properties are instantiated,

and:

which exact governed environment version exists for simulation execution.

Within the broader SIMULOS® lifecycle:

Simulation Object → Simulation Purpose & Scope → Reality Representation → Simulation Assumptions → Simulation Model & Environment → Inputs & Data → Scenario Design → Simulation Execution → Simulation Output Governance → Reality Correlation → Simulation Evidence → Validation Interface → Operational Reliance → Drift / Resimulation / Retirement

SEVM closes the Simulation Model & Environment governance space and establishes the controlled versioned configuration required before the architecture proceeds into simulation inputs, data, scenarios, and execution.

Foundational Principle

A simulation result cannot be fully reconstructed or meaningfully compared unless the materially relevant Simulation Environment Version that produced it remains identifiable.

Canonical Closing Statement

Simulation Environment Versioning Module completes the Simulation Model & Environment Standard by establishing the governed version-control layer through which materially distinct Simulation Environment states remain identifiable across simulation activity. Through environment baselines, version identity, composition, dependency tracking, lineage, change classification, compatibility, activation, freezing, branching, rollback, drift detection, execution binding, comparability, historical preservation, and point-in-time reconstruction, SEVM ensures that changes to the environment cannot silently alter simulation behavior while historical and current executions continue to be represented as if they occurred under the same governed simulation conditions.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>